

# CS101 – Session 2 Study Sheet: C++ Fundamentals

Marcus Birkenkrahe

February 6, 2026

## 1. What This Session Was About

- Review core C++ concepts needed for CS101
- Shift focus from **control flow** to **data and structure**
- Emphasis on:
  - precision
  - explicit decisions
  - disciplined program structure
- Foundation for arrays, containers, and data structures

## 2. Minimal Program Structure (Know This Cold)

You should recognize and explain each part:

- **#include**
  - Brings in library declarations (e.g. `<iostream>`)
  - Required before using library features
- **main()**
  - Program entry point
  - Execution starts here
- **Variables**

- Must be declared with a type
  - Should be initialized before use
- **Program flow**
  - Input → Computation → Output
- **Streams**
  - `cin` reads from standard input
  - `cout` writes to standard output

### 3. Core Concepts You Are Expected to Know

#### Variables & Types

- `int`, `double`, `const`
- Types are explicit in C++
- Initialization matters for correctness and clarity

#### Arithmetic

- Integer vs floating-point arithmetic
- Operator precedence applies
- Parentheses remove ambiguity

#### Output

- Mixing text and values requires care
- Formatting is part of correctness
- Precision affects readability and meaning

#### Constants

- Declared with `const`
- Represent values that must not change
- Make intent explicit

## **Type Conversion**

- Converting `double`  $\rightarrow$  `int` truncates the fractional part
- Information may be lost
- `static_cast<int>` makes intent explicit and safe

## **Conditionals**

- `if / else if / else`
- Conditions are mutually exclusive
- Exactly one branch executes

## **Loops**

- `for` loops repeat a block of code
- Loop variable has limited scope
- Common pattern: count from 0
- Off-by-one errors are common

## **Arrays**

- Fixed size once declared
- Indexing starts at 0
- Loops are the natural way to process arrays

## **Functions**

- Group a computation under a name
- Take parameters, return a value
- Improve clarity and reuse
- In C++, functions must be declared before use

## 4. Common Mistakes to Watch For

- Using uninitialized variables
- Printing expressions instead of stored results
- Mixing integer and floating-point types unintentionally
- Incorrect loop bounds
- Assuming user input is always valid
- Mixing input, computation, and output in one step

## 5. Conceptual Takeaways (Likely Test Material)

- Arithmetic and logic are stable across languages
- C++ forces decisions to be explicit
- Well-structured programs separate:
  - input
  - computation
  - output
- Correctness is necessary, but readability also matters

## 6. Self-Check Questions

You should be able to answer these without writing code:

- What happens when a `double` is converted to an `int`?
- Why store a computed result in a variable before printing?
- Why does array indexing start at 0?
- Why does C++ require more structure than Python?
- What kinds of errors can incorrect loop conditions cause?